



CIRCUIT BREAKER BEST PRACTICES

Last Updated: 05/25/2018

This guide discusses best practices for calling NDe services in peak traffic periods such as during a product launch. High-heat launches put an intense load on services and system resources. The goal of this document is to outline best practices to avoid putting further stress on system health from clients. In addition to the general recommendations listed in the [Service Call Best Practices](#) section, specific performance, retry and fallback best practices are listed by service.

Service Call Best Practices

Many factors can affect microservice performance and availability such as heavy network traffic and instance and database under scaling. Eureka instability also contributes to the problem by leaving services unable to know where to send requests. While these factors are not in the control of the service caller, there are actions that callers should take to help ensure system health.

Use the Circuit Breaker Pattern

Use the [Circuit Breaker Pattern](#) when calling other services (either internal or external) to avoid waiting indefinitely for a response from a non-responsive service and to provide fallback behavior for a service failure. [Hystrix](#) and [FastBreak](#) are examples of Circuit Breaker libraries currently used by Nike microservices.

Use the Exponential Backoff Retry Pattern

Unless otherwise noted, callers should follow the [Exponential Backoff Retry Pattern](#) to

determine how long to wait in between retries without modifying the request when the service returns a 429 or 5xx error. To use this pattern, a backoff increment value is used to calculate the wait time between retries. Wait time is calculated by wait time + backoff increment. For example, when the backoff increment is 100ms, the first four retry wait times are listed below.

- 1st retry: 100ms
- 2nd retry: 200ms
- 3rd retry: 400ms
- 4th retry: 800ms

Use Jitter

Clients should consider using [Jitter](#) to more randomly distribute retry calls to decrease load on the service.

Visit [API Error Patterns](#) for more information on retry recommendations.

Use Distributed Tracing

Many of Nike's microservices make calls to other services, quickly fanning out processing control. This complexity can make it difficult to troubleshoot bottlenecks and debug problems. Distributed tracing can help this situation by stepping through the round trip of a request and illuminating problems. [Wingtips](#) is the recommended distributed tracing tool.

Be Aware of Bot Rules

Bot rules are in place that block calls to these APIs by IP and upmid for a period of time when more than 300 calls per minute come through the public and edge routers. Service-to-service calls are not affected by these limits. For more information visit [Bot Monitoring and Mitigation](#).

Buy Service

Listed below are the best practices for calling each Buy service.

- [Carts](#)
- [Cart Reviews](#)
- [Shipping Options](#)
- [Checkout Preview](#)
- [Checkout Preview Job](#)
- [Checkout Submit](#)
- [Checkout Submit Job](#)
- [Launch Checkout Submit](#)

Carts

Endpoint: /buy/carts/v2/

Topic	Best Practice
Validation	Pass in all Checkout items and a valid two-digit ISO country. When updating an existing cart, ensure the request brand, channel and region matches the saved cart.
Performance	Multiple Checkout items may slow down the response because Carts validates each one. Regardless, always pass in all Checkout items. When updating the cart, use the PATCH method instead of PUT for best performance.
Circuit breaker trigger	Carts repeated call failure to the Merchandised Product, Merchandised

Topic	Best Practice
	Skus, Availability, Value-added service, Merchandised Price, Product Content and Exclusive Access services for validation can open the circuit.
Circuit breaker fallback behavior	None
Retry pattern for API callers	None
Fallback behavior for API callers	None

Cart Reviews

Endpoint: /buy/cart_reviews/v1/

Topic	Best Practice
Retry pattern for API callers	429 and 5xx error responses can be retried up to 6 times. Use the Exponential Backoff Retry Pattern, waiting 100ms between calls.
Fallback behavior for API callers	When the retry limit has been reached, callers can skip and proceed processing since Cart Reviews is not required for checkout.

Shipping Options

Endpoint: /buy/shipping_options/v2

Topic	Best Practice
Retry pattern for API callers	429 and 5xx error responses can be retried up to 6 times. Use the Exponential

Topic	Best Practice
	Backoff Retry Pattern, waiting 100ms between calls.
Fallback behavior for API callers	None

Checkout Preview

Endpoint: /buy/checkout_previews/v2

Topic	Best Practice
Retry pattern for API callers	429 and 5xx error responses can be retried up to 6 times. Use the Exponential Backoff Retry Pattern, waiting 100ms between calls.
Fallback behavior for API callers	When the retry limit has been reached, callers can skip and proceed to Checkout Submit since Checkout Preview is not required for checkout.

Checkout Preview Job

Endpoint: /buy/checkout_previews/v2/jobs/

Topic	Best Practice
Retry pattern for API callers	404, 429 and 5xx error responses can be retried up to 6 times. Use the Exponential Backoff Retry Pattern, waiting 100ms between calls.
Fallback behavior for API callers	When the retry limit has been reached, callers can skip and proceed to Checkout Submit since Checkout Preview is not

Topic	Best Practice
	required for checkout.

Checkout Submit

Endpoint: /buy/checkouts/v2/

Topic	Best Practice
Retry pattern for API callers	429 and 5xx error responses can be retried up to 6 times. Use the Exponential Backoff Retry Pattern, waiting 100ms between calls.
Fallback behavior for API callers	None

Checkout Submit Job

Endpoint: /buy/checkouts/v2/jobs/

Topic	Best Practice
Retry pattern for API callers	404, 429 and 5xx error responses can be retried up to 6 times. Use the Exponential Backoff Retry Pattern, waiting 100ms between calls.
Fallback behavior for API callers	None

Launch Checkout Submit

Endpoint: /buy/launch_checkouts/v2/

Topic	Best Practice
Retry pattern for API callers	429 and 5xx error responses can be retried up to 6 times. Use the Exponential Backoff Retry Pattern, waiting 100ms between calls.
Fallback behavior for API callers	None

Availability

Listed below are the best practices for calling each Availability service.

- [Product Inventory Availability](#)
- [SKU Availability](#)

Product Availability

Endpoint: /deliver/available_products/v1/

Topic	Best Practice
Performance	When calling the Product Availability List endpoint, send 5 productids in batch at a time.
Circuit breaker trigger	Product Availability's repeated call failure to the Merchandised Product service
Circuit breaker fallback behavior	None
Retry pattern for API callers	Use the Exponential Backoff Retry Pattern, waiting 100ms between calls. The caller should determine the retry limit.

Topic	Best Practice
Fallback behavior for API callers	Caller should default availability to false.

SKU Availability

Endpoint: /deliver/available_skus/v1/

Topic	Best Practice
Performance	When calling the Get SKU Availability Multi endpoint, send up to 25 skuids or 5 productids in batch at a time.
Circuit breaker trigger	Available SKUs' repeated call failure to the Merchandised Product service or Merchandised SKU service
Circuit breaker fallback behavior	None
Retry pattern for API callers	Use the Exponential Backoff Retry Pattern, waiting 100ms between calls. The caller should determine the retry limit.
Fallback behavior for API callers	Caller should default availability to false.

Launch Service

Coming soon

Merchandised Product Service

Listed below are the best practices for calling each Merchandised Product service.

- [Merchandised Product Caching](#)
- [Merchandised Product](#)
- [Merchandised Product SKUs](#)
- [Merchandised Product Prices](#)
- [Product Content](#)
- [Merchandised Value-added Services](#)

Merchandised Product Caching

Because product and SKU information does not change frequently, service-to-service calls made to the Merchandised Product service should use distributed caching. Pre-loading of the cache prior to launch is recommended so no customer has a degraded shopping experience while the service loads the data into its cache. During launch, retrieve the merchandised product data from cache if available rather than calling the service.

Experiences calling the Merchandised Product services directly should not cache these endpoints. Rather, if the Cache-Control header is set, the browser will cache product data for that time period.

Merchandised Product

Endpoint: /merch/products/v2/

Topic	Best Practice
Performance	If you have one product UUID, call the <code>Merchandised Product by ID</code> endpoint. If you have a list of product UUIDs, call the <code>Merchandised Product List</code> endpoint with the <code>id</code> filter to list the products in batch.

Topic	Best Practice
	When filtering by id, request 25 ids or less at a time.
Retry pattern for API callers	Use the Exponential Backoff Retry Pattern. The caller should determine the wait time between calls and retry limit.
Fallback behavior for API callers	None

Merchandised Product SKUs

Endpoint: /merch/skus/v2/

Topic	Best Practice
Performance	If have one SKU UUID, call the Merchandised Product SKU by ID endpoint. If you have a list of SKU UUIDs, call the Merchandised Product SKU List endpoint with the id filter to list the SKUs in batch. When filtering by id, request 25 ids or less at a time.
Retry pattern for API callers	Use the Exponential Backoff Retry Pattern. The caller should determine the wait time between calls and retry limit.
Fallback behavior for API callers	None

Merchandised Product Prices

Endpoint: /merch/prices/v2/

Topic	Best Practice
Performance	If you have one price UUID, call the Merchandised Product Prices by ID endpoint. If you have a list of price UUIDs, call the Merchandised Product Prices List endpoint with the id filter to list the prices in batch. When filtering by id, request 25 ids or less at a time.
Retry pattern for API callers	Use the Exponential Backoff Retry Pattern. The caller should determine the wait time between calls and retry limit.
Fallback behavior for API callers	None

Product Content

Endpoint: /merch/contents/v1/

Topic	Best Practice
Performance	If you need the content or images for only one product, call a single product endpoint with the style-color. When calling a multiple product endpoint, request 25 style-colors or less in the request at a time.
Retry pattern for API callers	Use the Exponential Backoff Retry Pattern. The caller should determine the wait time between calls and retry limit.
Fallback behavior for API callers	None

Merchandised Value-added Services

Endpoint: /merch/value_added_services/v1/

Topic	Best Practice
Performance	If you have one product UUID, call the Merchandised Value Added Services by ID endpoint. When calling the Merchandised Value Added Services List endpoint filtering by ID, send 25 IDs or less in the request at a time.
Retry pattern for API callers	Use the Exponential Backoff Retry Pattern. The caller should determine the wait time between calls and retry limit.
Fallback behavior for API callers	None

Payment Service

Listed below are the best practices for calling each Payment service.

- [Payment Options](#)
- [Payment Stored Payments](#)
- [Payment Preview](#)
- [Payment Approval](#)
- [Payment Credit Card Submit](#)
- [Payment Apple Pay](#)
- [Payment Wallet](#)
- [Payment Deferred Payment](#)

Payment Options

Endpoints: /payment/options/v2/, /payment/validate_payments/v2/

Topic	Best Practice
Validation	Pass in all Checkout items when available.
Performance	Multiple Checkout items may slow down the response because Payment Options validates each one. Regardless, pass in all Checkout items when available.
Circuit breaker trigger	Payment Option's repeated call failure to the Merchandised Product Service for Checkout item validation.
Circuit breaker fallback behavior	Payment Options assumes the Checkout item product type is inline for validation purposes and continues processing.
Retry pattern for API callers	429 responses can be retried twice. Wait the amount of time sent in the Retry-After header between calls.
Fallback behavior for API callers	When a caller reaches the retry limit, it can default to a non-stored credit card as a payment option in all countries except China.

Payment Stored Payments

Endpoint: /consumer/storedpayments/

Topic	Best Practice
Validation	When available, always pass in the

Topic	Best Practice
	shipping address to the <code>FETCH SAVED PAYMENTS FOR A UPID</code> endpoint to determine if the customer must validate the stored credit card's CVV before submitting the order.
Performance	When gift card balance is not needed, set the <code>includeBalance</code> flag to false so the stored gift card balance is not retrieved when gathering the customer's stored payments.
Circuit breaker triggers	Payment Stored Payments' repeated call failure to the Payment Gift Card service when saving a Gift Card or retrieving the balance Payment PayPal service when saving a new PayPal payment type to the customer's profile Payment Cybersource service trying to store or update a customer's credit card.
Circuit breaker fallback behavior	None
Retry pattern for API callers	429 responses can be retried twice. Wait the amount of time sent in the <code>Retry-After</code> header between calls.
Fallback behavior for API callers	None

Payment Preview

Endpoint: `/payment/preview/v2`

Topic	Best Practice
Performance	When a customer selects to pay by stored credit card, check the validateCVV flag on the response from the Stored Payments Service. If the value is true, allow the customer to verify their CVV number in your experience and send it to the Payment Credit Card Submit service. Otherwise, Payment Preview will fail due to an unverified CVV number.
Circuit breaker trigger	Payment Preview's repeated call failure to the Payment Gift Card service when retrieving the balance Stored Payment service when retrieving the customer's stored payment details Credit Card Submit service when validating the credit card info id
Circuit breaker fallback behavior	None
Retry pattern for API callers	429 responses can be retried twice. Wait the amount of time sent in the Retry-After header between calls.
Fallback behavior for API callers	None

Payment Approval

Endpoint: /payment/approval/v2/

Topic	Best Practice
Circuit breaker trigger	Payment Approval calls several Cloud service endpoints, many of which call third party systems. Repeated call failure

Topic	Best Practice
	to any of these services triggers the circuit breaker
Circuit breaker fallback behavior	Payment Approval marks the fraud decision as “unknown” so fraud scoring is retried downstream.
Retry pattern for API callers	429 responses can be retried twice. Wait the amount of time sent in the Retry-After header between calls.
Fallback behavior for API callers	None

Payment Credit Card Submit

Endpoint: /services/, /creditcardsubmit/

Topic	Best Practice
Performance	If the customer is not required to supply credit card information or cvv, do not call a Credit Card Submit endpoint that loads the credit card iFrame.
Retry pattern for API callers	429 responses can be retried twice. Wait the amount of time sent in the Retry-After header between calls.
Fallback behavior for API callers	None

Payment Apple Pay

Endpoint: /payment/applepay_sessions/v2/

Topic	Best Practice
Circuit breaker trigger	Payment Apple Pay's repeated call failure to the Apple gateway to start the ApplePay web session
Circuit breaker fallback behavior	None
Retry pattern for API callers	429 responses can be retried twice. Wait the amount of time sent in the Retry-After header between calls.
Fallback behavior for API callers	None

Payment Wallet

Endpoint: /payment/paypal_details/, /payment/paypal_express/v1/, /payment/paypal_mark/v1/

Topic	Best Practice
Circuit breaker trigger	Payment Wallet calls other Cloud service endpoints, some of which call third party systems. Repeated call failure to these external services triggers the circuit breaker.
Circuit breaker fallback behavior	None
Retry pattern for API callers	429 responses can be retried twice. Wait the amount of time sent in the Retry-After header between calls.
Fallback behavior for API callers	None

Payment Deferred Payment

Endpoint: /payment/deferred_wechat_payments/v1/, /payment/deferred_payment_forms/

v1/, /payment/deferred_payment_status/v1/

Topic	Best Practice
Circuit breaker trigger	Payment Deferred Payment calls other Cloud service endpoints, many of which call third party systems. Repeated call failure to these external services triggers the circuit breaker.
Circuit breaker fallback behavior	None
Retry pattern for API callers	429 responses can be retried twice. Wait the amount of time sent in the Retry-After header between calls.
Fallback behavior for API callers	None

Product Feeds Service

Listed below are the best practices for calling each Product Feeds Service.

- [Product Feed V2](#)
- [Product Feed Rollups V2](#)
- [Product Feed Exclusive Threads V2](#)

Product Feed V2

Endpoint: /product_feed/threads/v2

Topic	Best Practice
Circuit breaker trigger	None
Circuit breaker fallback behavior	None
Retry pattern for API callers	Use the Exponential Backoff Retry

Topic	Best Practice
	Pattern. The caller should determine the wait time between calls and retry limit.
Fallback behavior for API callers	None

Product Feed Rollups V2

Endpoint: /product_feed/rollup_threads/v2

Topic	Best Practice
Circuit breaker trigger	Product Feed Rollups calls Smart Search. Repeated call failure to this service triggers the circuit breaker.
Circuit breaker fallback behavior	None
Retry pattern for API callers	Use the Exponential Backoff Retry Pattern. The caller should determine the wait time between calls and retry limit.
Fallback behavior for API callers	None

Product Feed Exclusive Threads V2

Endpoint: /product_feed/exclusive_threads/v2

Topic	Best Practice
Circuit breaker trigger	None
Circuit breaker fallback behavior	None
Retry pattern for API callers	Use the Exponential Backoff Retry Pattern. The caller should determine the wait time between calls and retry limit.

Topic	Best Practice
Fallback behavior for API callers	None

Related Links

[NDe Docs Home](#)

[Get Started](#)